

Roteiro — Aula 3: as rotas de autenticação

Deck: `slides/build/aula-03-autenticacao.pptx` (57 slides) **Apostila da turma:** `03-autenticacao.md`

• **Checkpoint:** `aula-03`

Esta é a aula que não cabe. São 1396 linhas de código em 37 arquivos. Digitando, dá 6h30. Leia a seção seguinte antes de qualquer outra coisa.

A decisão que você precisa tomar antes

Você tem duas formas de conduzir esta aula:

Opção A — código pronto, leitura guiada (recomendada)

No começo da aula, todo mundo copia o código do gabarito para **o próprio projeto**:

```
cd ~/capacitacao-gabarito && git checkout aula-03
rsync -a app config db test Gemfile Gemfile.lock ~/automic_auth_api/
cd ~/automic_auth_api && bundle install && bin/rails db:migrate && bin/rails test
```

O `Gemfile` vai junto porque a Aula 3 acrescenta `jwt`, `rack-cors` e `letter_opener` — sem ele a aplicação nem sobe. Os 71 testes verdes são a prova de que a cópia deu certo. **Só depois disso** você percorre os arquivos projetados, explicando decisão por decisão, com eles acompanhando no editor.

A prática vira **modificar** o que já existe, que é o que se faz num emprego de verdade. E o código fica no repositório deles, que é de onde a Aula 4 vai fazer o deploy.

Cabe em ~2h50. É o que este roteiro assume.

Opção B — digitar junto

Só funciona se você tiver 6h ou dividir em dois encontros. Se for por aqui, corte para **três rotas**: cadastro, login e logout. Recuperação de senha vira leitura da apostila.

Antes de começar

- [] O `rsync` do gabarito testado num projeto limpo, com `bin/rails test` verde depois.
- [] Servidor rodando e o Insomnia com as cinco requisições já montadas — você vai fazer o fluxo completo ao vivo duas vezes.

- [] jwt.io aberto numa aba.
- [] Um token válido copiado, pronto para colar no jwt.io.
- [] `tmp/letter_opener/` limpo, para o e-mail da demo ser o primeiro da lista.
- [] Editor com `app/services/` e `app/controllers/api/v1/` já abertos na barra lateral.

Cronograma (Opção A)

Relógio	Slides	Bloco	Min
00:00	1–3	Abertura e o caminho de uma requisição	6
00:06	4–13	Arquitetura: service, serializer, erro, filtro, middleware	28
00:34	14–16	Cadastro e enumeração de usuários	12
00:46	17–20	Mailer, letter_opener, <code>deliver_now</code>	14
01:00	21–28	Sessão, JWT, Base64, assinatura	26
01:26	—	Intervalo	10
01:36	29–36	Login, cookie, XSS, CSRF, 401×403	28
02:04	37–43	Logout: denylist e <code>token_version</code>	24
02:28	44–49	Recuperação de senha e a transação	18
02:46	50–54	Testes, ferramentas, CORS	15
03:01	55	Prática	25
03:26	56–57	Recapitulação e fim	4

Dá 3h30 mesmo na Opção A. Cortes, em ordem:

1. Slides 33–34 (XSS e CSRF) — o material fica na apostila. **–8 min**
2. Slides 53–54 (ferramentas e CORS) — mostre rodando e siga. **–8 min**
3. Slides 17–20 (mailer) — mostre só o e-mail abrindo no navegador. **–8 min**

Nunca corte o bloco 37–43. O logout é o ponto alto da aula, e é o que diferencia esta capacitação de um tutorial de YouTube.

Bloco a bloco

00:00 — O caminho (1-3)

O slide 3 é o mapa da aula inteira. Deixe ele na tela enquanto fala a regra:

Controller recebe requisição e devolve resposta. Regra de negócio não mora nele.

00:06 — Arquitetura (4-13)

Abra `app/services/users/register.rb` projetado e conte as seis coisas que ele faz. Depois pergunte: *"quanto disso vocês conseguiriam testar sem subir uma requisição HTTP inteira?"*

- **7** — serializer. Faça a demonstração do susto: *"o que acontece se eu escrever `render json: user`?"*
Mostre no console:

```
ruby User.first.as_json.keys
```

Está lá o `password_digest` e os digests dos códigos. **Uma linha, e vazou.**

- **10-12** — `before_action` e middleware. O slide 12 é o mapa completo. Se der, rode:

```
bash bin/rails middleware
```

- **13** — strong parameters. A frase: *"sem isso, alguém manda `role: admin` no cadastro e vira admin. Isso tem nome: mass assignment, e já derrubou sistema grande."*

00:34 — Cadastro (14-16)

O slide 16 é o primeiro dos quatro momentos de "enumeração" da aula. Marque isso: você vai voltar nele três vezes, e no fim eles devem conseguir prever a decisão sozinhos.

Pergunta para a turma antes de virar o slide: *"o e-mail já existe. O que a API deve responder?"* Alguém vai dizer "este e-mail já está cadastrado". Aí você mostra por que não.

00:46 — E-mail (17-20)

Faça o cadastro ao vivo no Insomnia e **mostre o e-mail abrindo no navegador** pelo `letter_opener`. É um daqueles momentos em que a turma acorda.

No slide 20, a decisão contra o livro-texto: `deliver_now` porque, numa fila, o código de 6 dígitos ficaria gravado **em texto** na tabela de jobs.

01:00 — JWT (21-28)

O bloco conceitual mais denso. Comece pelo problema (22): *"lembram que HTTP não tem memória? Então como o servidor sabe que vocês já entraram?"*

Faça a demo do jwt.io. Cole o token que você preparou e mostre o payload legível na tela.

"Está assinado. Não está criptografado. Qualquer um lê. Então nunca ponham aqui nada que não possa ser lido."

No **28**, o `true` do `JWT.decode`. Diga que já foi CVE em várias bibliotecas, e que eles vão brincar com isso na prática.

01:36 — Login (29–36)

- **30** — os dois transportes. O `client` no corpo existe por isso.
- **31–32** — o que é cookie, e as três flags.
- **33–34** — XSS e CSRF. Se o tempo apertar, é aqui que você corta.
- **36** — o segundo "enumeração": mesma mensagem para e-mail inexistente e senha errada. E o `DUMMY_PASSWORD_DIGEST` da Aula 2 volta, fechando o buraco pelo lado do tempo.

02:04 — Logout (37–43)

O ponto alto. Conduza como um problema, não como uma solução.

1. Pergunte: "o token vale 24h e o servidor não guarda sessão. O que 'sair' significa?"
2. Deixe a turma propor. Alguém vai dizer "apaga no cliente". Aceite e pergunte: "e se alguém já copiou o token?"
3. Aí você apresenta a denylist (40).
4. O slide 41 é a sacada: o TTL de cada entrada é o que faltava para o token expirar, então **a lista se limpa sozinha**.
5. E o 42 mostra a outra ferramenta: `token_version` derruba tudo de uma vez.

Demonstre ao vivo. Vale mais que os seis slides:

```
TOKEN=... # pegue do login
curl $API/me -H "Authorization: Bearer $TOKEN" # 200
curl -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl -i $API/me -H "Authorization: Bearer $TOKEN" # 401
```

02:28 — Recuperação de senha (44–49)

Terceiro e quarto "enumeração" (46). A essa altura, **pergunte antes**: "o e-mail não existe. O que respondemos?" Eles devem acertar sozinhos.

O slide 48 amarra com a Aula 2: a transação. E o 49 tem o ponto do fluxo inteiro —

`invalidate_sessions!`: "se alguém invadiu a conta, trocar a senha tem que expulsar o invasor. Sem essa linha, ele continua logado."

Demonstre: faça o reset e mostre o token antigo virando 401.

03:01 — Prática (55)

Na Opção A, a prática é **modificar**:

1. Fluxo completo no Insomnia (obrigatório).
2. Colar o próprio token no jwt.io e achar `sub`, `jti` e `exp`.
3. **Bônus 1:** `PATCH /api/v1/me` para editar só o `name`.
4. **Bônus 2:** trocar o `true` do `JWT.decode` por `false`, forjar um token com outro segredo, ver `/me` responder 200 — e depois desfazer. É o exercício que mais marca.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Se o payload é público, não é inseguro?"	Não, desde que você não coloque segredo lá. O token prova quem você é; ele não precisa esconder isso de você.
"Por que não usar sessão como todo mundo?"	Porque o cliente principal é app nativo, que não tem cookie. E porque assim qualquer servidor valida sozinho, sem sessão compartilhada.
"Então JWT é pior que sessão?"	É diferente. Sessão facilita o logout e complica o escalar; token faz o contrário. Nós resolvemos o logout com a denylist.
"A denylist não é uma sessão disfarçada?"	Um pouco — mas só guarda os tokens revogados, não todos, e expira sozinha. É bem mais barata.
"Por que 15 minutos no código, e não 1 hora?"	Janela curta reduz o tempo em que um código interceptado serve. 15 min é o suficiente para checar e-mail sem correr.
"E se o e-mail do usuário estiver errado no cadastro?"	Ele nunca confirma, nunca loga, e a conta fica órfã. É por isso que existe o <code>resend</code> .
"Posso usar Devise em vez de escrever tudo isso?"	Pode, e em projeto de verdade normalmente é o que se faz. Mas você não entenderia nada do que ele faz — e é isso que estamos aprendendo.
"Por que <code>unprocessable_content</code> e não <code>unprocessable_entity</code> ?"	Mesmo código 422. O nome foi renomeado na especificação, e o Rails 8 seguiu.

Dever de casa

1. Fazer os dois bônus.
2. Ler `app/services/users/complete_password_reset.rb` inteiro e explicar, por escrito, por que a política de senha é validada **antes** de consumir o código.
3. Rodar `bin/brakeman` e ler o que ele procura.